LAPORAN PRAKTIKUM
Struktur Data

MODUL 4
(Singly Linked List)



Disusun oleh:

Rezky Farrel
109082500203
S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
TAHUN 2026

Guided

1. Guided 1

Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = NULL;
}

address alokasi(infotype x) {
    address P = new elmlist;
    P->info = x;
    P->next = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

void insertFirst(List &L, address P) {
    P->next = L.first;
    L.first = P;
}

void insertLast(List &L, address P) {
    if (L.first == NULL) {
        insertFirst(L, P);
    } else {
        address Q = L.first;
        while (Q->next != NULL) {
            Q = Q->next;
        }
        Q->next = P;
    }
}
```

```
void insertAfter(List &L, address P, address Prec) {
    if (Prec != NULL) {
        P->next = Prec->next;
        Prec->next = P;
    }
}

void deleteFirst(List &L, address &P) {
    if (L.first != NULL) {
        P = L.first;
        L.first = P->next;
        P->next = NULL;
    }
}

void deleteLast(List &L, address &P) {
    if (L.first != NULL) {
        if (L.first->next == NULL) {
            deleteFirst(L, P);
        } else {
            address Q = L.first;
            while (Q->next->next != NULL) {
                Q = Q->next;
            }
            P = Q->next;
            Q->next = NULL;
        }
    }
}

void deleteAfter(List &L, address &P, address Prec) {
    if (Prec != NULL && Prec->next != NULL) {
        P = Prec->next;
        Prec->next = P->next;
        P->next = NULL;
    }
}

void printInfo(List L) {
    address P = L.first;
    while (P != NULL) {
        cout << P->info << ' ';
        P = P->next;
    }
    cout << '\n';
}

int main() {
    List L;
    address P1, P2, P3, P4, P5;
```

```
    createList(L);

    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

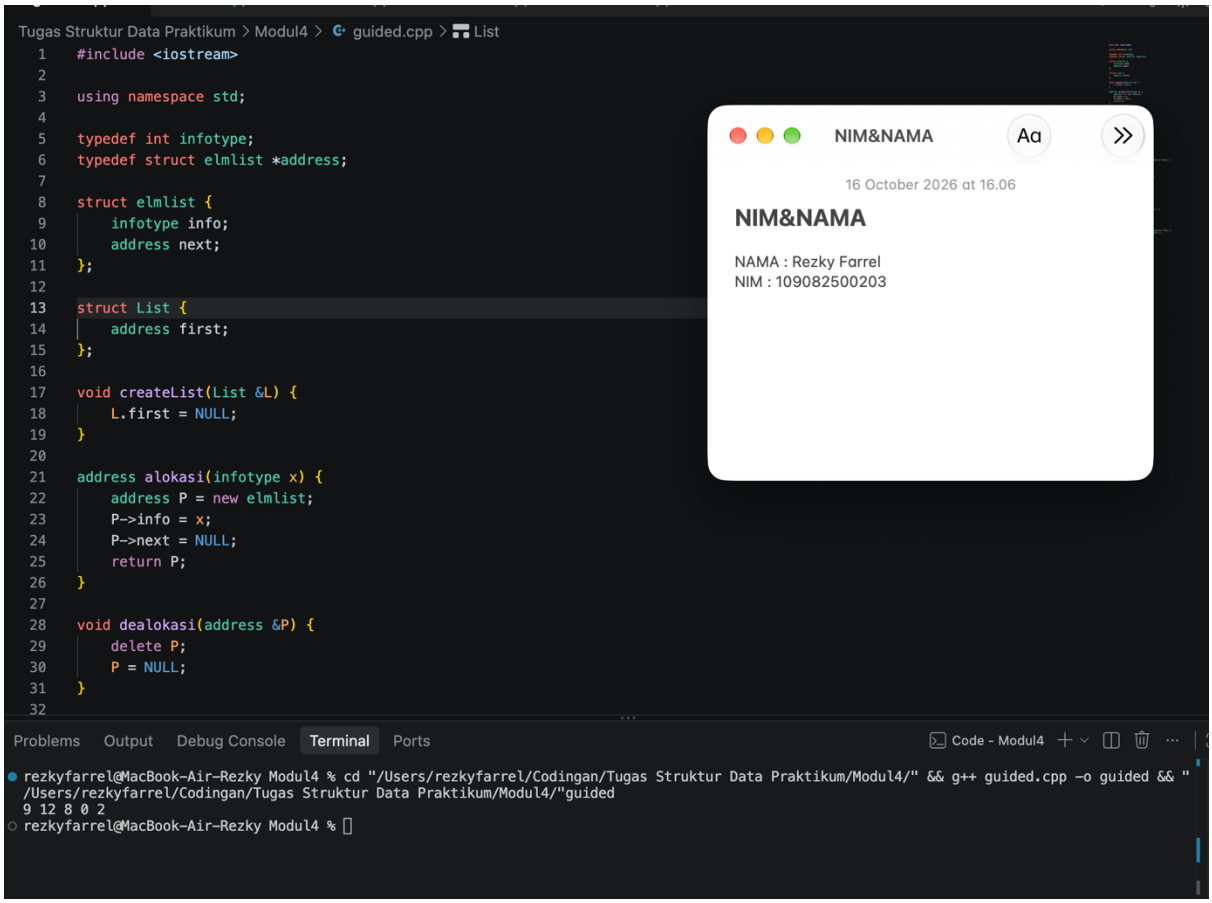
    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    printInfo(L);

    return 0;
}
```

Screenshoot program



LATIHAN SOAL MODUL

Link Repo Github Modul 4 : https://github.com/RezkyFarrel1/Struktur-Data/tree/main/Modul%204

1. **Soal 1**

   Source Code

```cpp
#include <iostream>
#include <string>

using namespace std;

struct Node {
    string nama;
    Node* next;
};

void insertLast(Node*& head, Node*& tail, string nama) {
    Node* newNode = new Node();
    newNode->nama = nama;
    newNode->next = nullptr;

    if (head == nullptr) {

        head = tail = newNode;
    } else {

        tail->next = newNode;
        tail = newNode;
    }
}

void deleteFirst(Node*& head, Node*& tail) {
    if (head == nullptr) {
        cout << "Antrean kosong." << endl;
        return;
    }

    Node* temp = head;
    cout << "Melayani: " << temp->nama << endl;

    head = head->next;

    if (head == nullptr) {
        tail = nullptr;
    }

    delete temp;
}
```

```
void printQueue(Node* head) {
    if (head == nullptr) {
        cout << "Antrean kosong." << endl;
        return;
    }

    cout << "Antrean: ";
    Node* current = head;
    while (current != nullptr) {
        cout << current->nama;
        if (current->next != nullptr) {
            cout << " -> ";
        }
        current = current->next;
    }
    cout << endl;
}

int main() {
    Node* head = nullptr;
    Node* tail = nullptr;

    int pilihan;
    string nama;

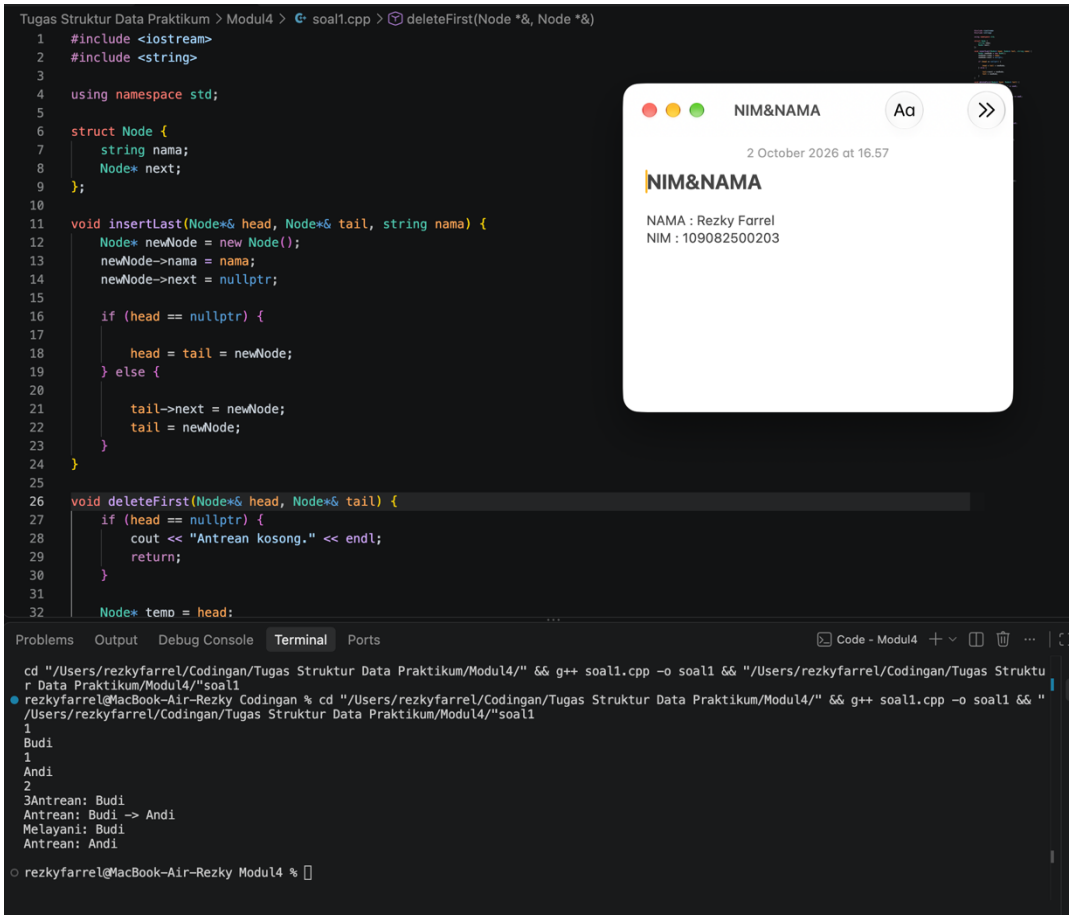
    while (true) {
        cin >> pilihan;

        if (pilihan == 1) {
            cin >> nama;
            insertLast(head, tail, nama);
            printQueue(head);
        } else if (pilihan == 2) {
            deleteFirst(head, tail);

            if (head != nullptr) {
                printQueue(head);
            }
        } else if (pilihan == 3) {
            break;
        }
    }

    return 0;
}
```

Screenshoot program



Deskripsi program

Program di atas menggunakan pointer head dan tail untuk mengelola antrean (Queue) dengan basis struktur Singly Linked List, sehingga mempermudah operasi insertLast (O(1)) dan deleteFirst (O(1)). Input didesain terus membaca angka hingga pengguna memasukkan pilihan 3 untuk keluar dari perulangan.

2. Soal 2

Source Code

```
#include <iostream>

using namespace std;

struct Node {
    int data;
    Node* next;
};

void insertLast(Node*& head, Node*& tail, int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = nullptr;

    if (head == nullptr) {
        head = tail = newNode;
```

```
    } else {
        tail->next = newNode;
        tail = newNode;
    }
}

void insertAfter(Node* head, int searchData, int newData) {
    Node* current = head;

    while (current != nullptr) {
        if (current->data == searchData) {

            Node* newNode = new Node();
            newNode->data = newData;

            newNode->next = current->next;
            current->next = newNode;

            return;
        }
        current = current->next;
    }
}

void printList(Node* head) {
    Node* current = head;
    while (current != nullptr) {
        cout << current->data;
        if (current->next != nullptr) {
            cout << " -> ";
        }
        current = current->next;
    }
    cout << endl;
}

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Node* head = nullptr;
    Node* tail = nullptr;

    for (int i = 0; i < n; ++i) {
        int val;
        cin >> val;
        insertLast(head, tail, val);
    }

    int searchData, newData;
```

```
    cin >> searchData;

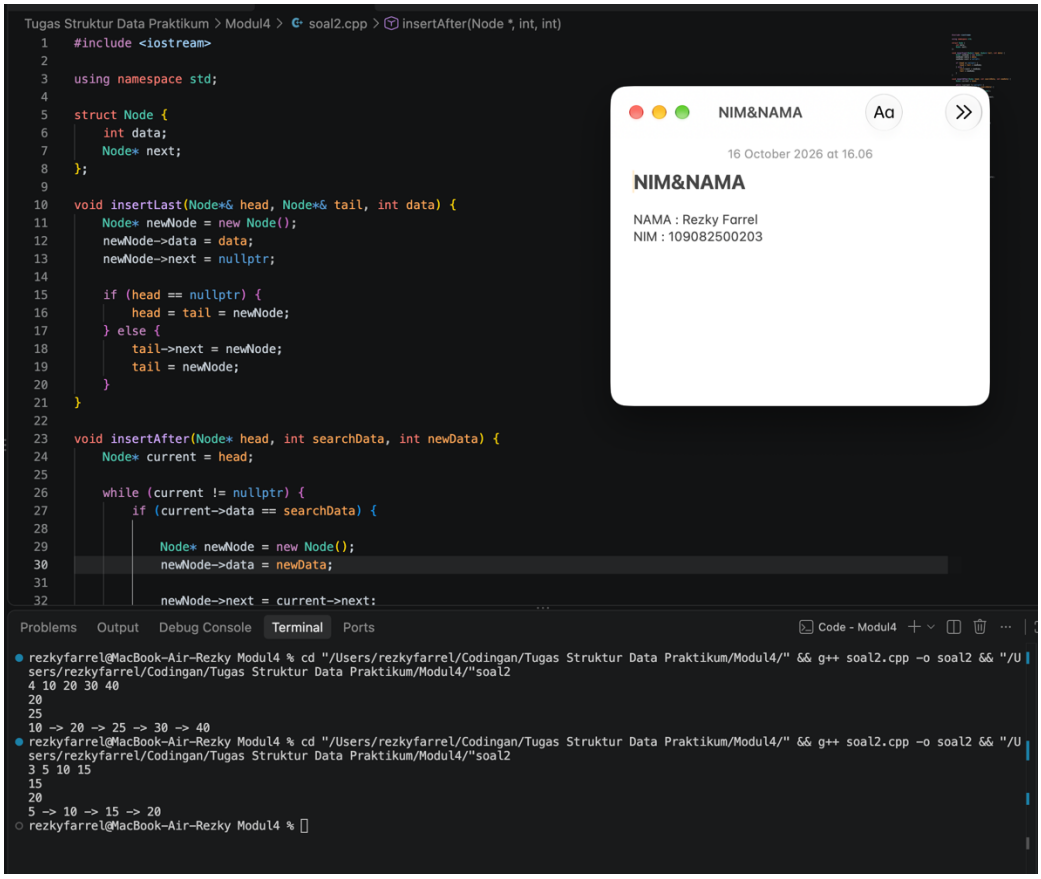
    cin >> newData;

    insertAfter(head, searchData, newData);

    printList(head);

    return 0;
}
```

Screenshoot program



Deskripsi program

Program di atas membaca jumlah elemen awal terlebih dahulu, lalu menggunakan perulangan untuk membangun struktur linked list awal. Setelah itu, program mengambil dua input berikutnya (nilai yang dicari dan nilai baru), mencari lokasi pointer nilai tersebut di dalam linked list, lalu menggeser pointer next untuk menyisipkan node baru tepat setelah data yang ditemukan tersebut.

3. **Soal 3**

Source Code

```
#include <iostream>

using namespace std;

struct Node {
```

```
    int data;
    Node* next;
};

void insertLast(Node*& head, Node*& tail, int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = nullptr;

    if (head == nullptr) {
        head = tail = newNode;
    } else {
        tail->next = newNode;
        tail = newNode;
    }
}

int calculateSum(Node* head) {
    int sum = 0;
    Node* current = head;

    while (current != nullptr) {
        sum += current->data;
        current = current->next;
    }

    return sum;
}

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Node* head = nullptr;
    Node* tail = nullptr;

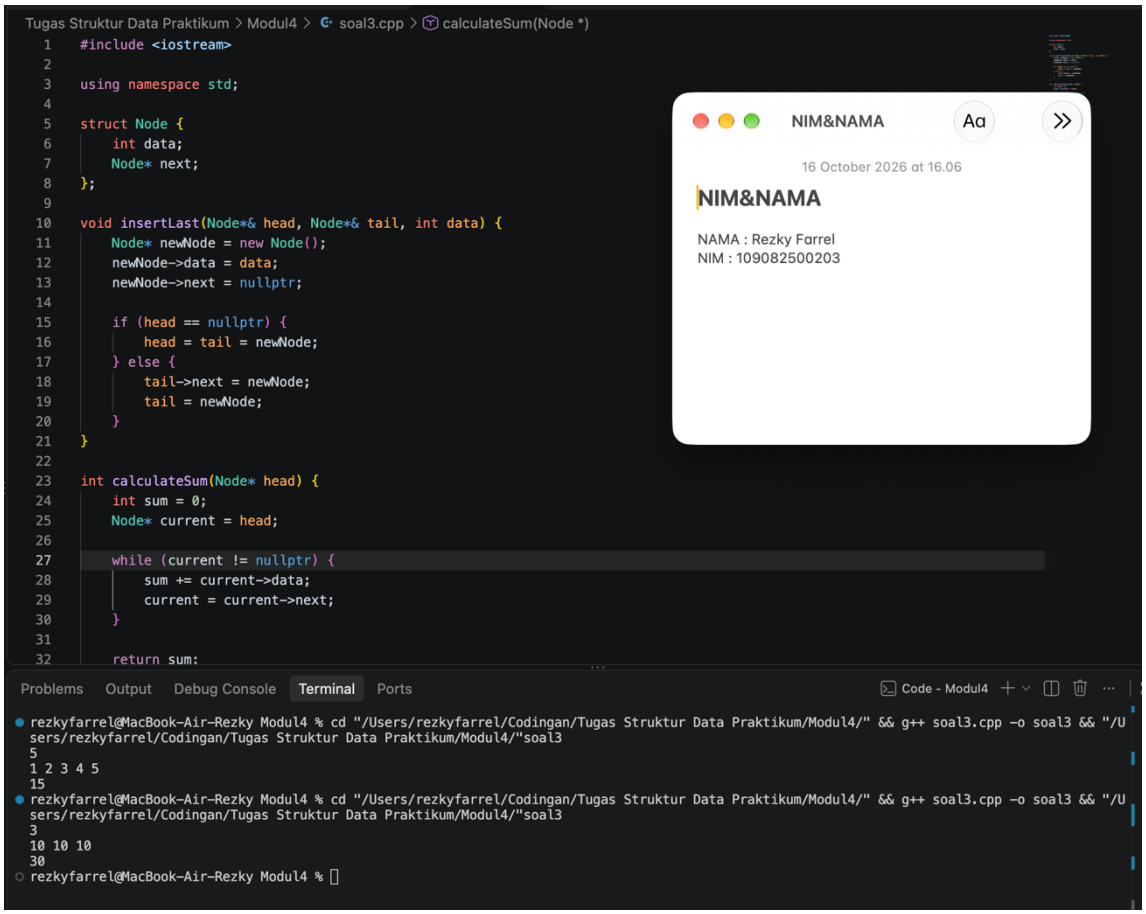
    for (int i = 0; i < n; ++i) {
        int val;
        cin >> val;
        insertLast(head, tail, val);
    }

    cout << calculateSum(head) << endl;

    Node* temp;
    while (head != nullptr) {
        temp = head;
        head = head->next;
        delete temp;
    }
```

```
    return 0;
}
```

Screenshoot program



Deskripsi program

Program di atas menggunakan perulangan while di dalam fungsi calculateSum untuk menelusuri (traversal) setiap node dari head hingga mencapai ujung (nullptr). Pada setiap iterasi, nilai data dari node saat ini ditambahkan ke dalam variabel sum. Hasil akhir sum kemudian dikembalikan dan dicetak ke layar

4. Soal 4

Source Code

```
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

struct Node {
    string judul;
    int tahun;
    Node* next;
};
```

```
class DaftarBuku {
private:
    Node* head;
    Node* tail;

public:
    DaftarBuku() : head(nullptr), tail(nullptr) {}

    ~DaftarBuku() {
        while (head != nullptr) {
            Node* temp = head;
            head = head->next;
            delete temp;
        }
    }

    // Menambah buku di urutan terakhir
    void insertLast(const string& judul, int tahun) {
        Node* baru = new Node{judul, tahun, nullptr};
        if (head == nullptr) {
            head = tail = baru;
        } else {
            tail->next = baru;
            tail = baru;
        }
    }

    // Menghapus buku di urutan terakhir
    void deleteLast() {
        if (head == nullptr) {
            return; // list kosong
        }
        if (head == tail) { // hanya satu data
            delete head;
            head = tail = nullptr;
            return;
        }
        Node* bantu = head;
        while (bantu->next != tail) {
            bantu = bantu->next;
        }
        delete tail;
        tail = bantu;
        tail->next = nullptr;
    }

    // Menampilkan seluruh buku
    void tampilkan() const {
        cout << "Daftar Buku:" << endl;
        Node* bantu = head;
        while (bantu != nullptr) {
```

```
            cout << bantu->judul << " (" << bantu->tahun << ")" << endl;
            bantu = bantu->next;
        }
    }
};

int main() {
    DaftarBuku list;
    string baris;

    while (getline(cin, baris)) {
        if (baris.empty()) continue;

        // Jika baris hanya berisi angka (nilai N), lewati
        bool hanyaAngka = true;
        for (char c : baris) {
            if (!isdigit(c)) { hanyaAngka = false; break; }
        }
        if (hanyaAngka) continue;

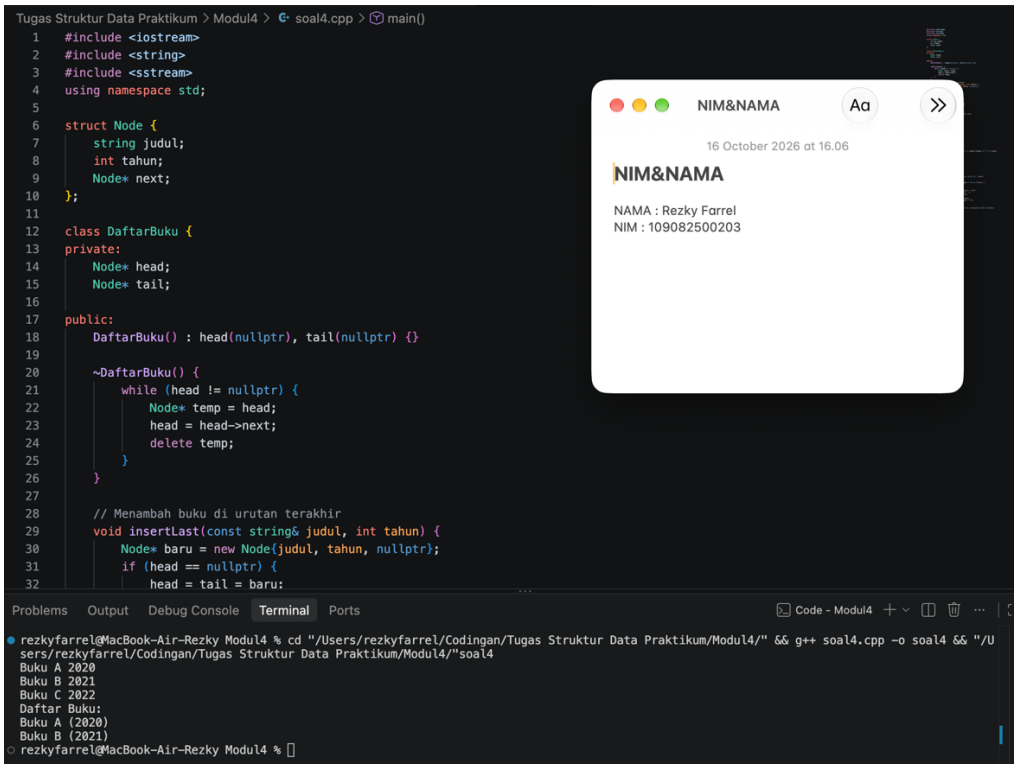
        // Token terakhir = tahun, sisanya = judul
        size_t pos = baris.find_last_of(' ');
        if (pos == string::npos) continue;

        string judul = baris.substr(0, pos);
        int tahun = stoi(baris.substr(pos + 1));
        list.insertLast(judul, tahun);
    }

    list.deleteLast(); // satu kali operasi penghapusan data terakhir
    list.tampilkan();

    return 0;
}
```

Screenshoot program



Deskripsi program

P rogram ini merupakan implementasi Singly Linked List dalam C++ untuk mengelola data buku yang menyimpan informasi judul dan tahun terbit menggunakan memori dinamis berwujud struct Node. Pada fungsi main, program secara terus-menerus membaca masukan (hingga End-Of-File atau EOF) yang diasumsikan selalu terdiri dari dua kata untuk judul dan satu bilangan bulat untuk tahun, lalu menyisipkannya ke bagian akhir antrean menggunakan fungsi insertLast secara efisien melalui pemanfaatan pointer head dan tail. Setelah keseluruhan data awal selesai dirangkai, program memanggil fungsi deleteLast untuk menghapus buku pada urutan paling belakang. Karena struktur yang digunakan adalah Singly Linked List yang tidak memiliki pointer mundur (seperti prev), proses penghapusan tersebut harus menelusuri (traversal) list dari elemen pertama (head) untuk mencari elemen tepat sebelum tail, menghapus node terakhir dari memori, dan memperbarui letak tail pada node sebelum penutup yang baru. Proses kemudian diakhiri dengan eksekusi fungsi printList yang menelusuri sisa elemen untuk mencetak daftar buku ke layar dengan format judul diikuti tahun terbit di dalam tanda kurung.